RISC V Components

> what are the components of RISC-V single cycle architecture

A **RISC-V single-cycle processor** executes **one instruction completely in one clock cycle**. The basic architecture can be broken into these components:

🧩 Main components

```
              ┌──────────────────┐
              │ Program Counter  │
              │       (PC)       │
              └────────┬─────────┘
                       ▼
              ┌──────────────────┐
              │Instruction Memory│
              └────────┬─────────┘
                  Instruction
                       ▼
              ┌──────────────────┐
              │   Control Unit   │
              └────────┬─────────┘
         ┌─────────────┼─────────────┐
         ▼             ▼             ▼
    ALU Control     RegWrite     MemRead/Write
         │
         ▼
   ┌──────────────┐
   │Register File │◄──────────────┐
   │   x0–x31     │               │
   │ 32 × 32-bit  │               │
   └──────┬───────┘               │
     rs1  │  rs2                  rd
          ▼                       │
     ┌─────────┐                  │
     │   ALU   │──────────────────┘
     └────┬────┘
          ├──────────► Data Memory
          │                 │
          │                 ▼
          │             Read Data
          ▼                 │
     ┌─────────┐            │
     │   MUX   │◄───────────┘
     └────┬────┘
          ▼
    Register File
```

For a basic **RV32I single-cycle CPU**, I'd divide it into the following blocks.

#	Component	Purpose
1	**Program Counter (PC)**	Holds address of current instruction
2	**PC + 4 Adder**	Calculates next sequential instruction
3	**Instruction Memory**	Stores and outputs instructions
4	**Instruction Decoder**	Extracts opcode, `rd`, `rs1`, `rs2`, funct fields
5	**Control Unit**	Generates control signals
6	**Register File**	32 × 32-bit registers
7	**Immediate Generator**	Generates sign-extended immediates
8	**ALU**	Arithmetic, logical and comparison operations
9	**ALU Control**	Determines which ALU operation to perform
10	**Data Memory**	Handles `LW`/`SW`
11	**Branch/Jump Logic**	Determines whether PC changes conditionally
12	**MUXes**	Select between different data paths
13	**PC Target Adder**	Calculates branch/jump target
14	**Write-back logic**	Selects ALU result or memory data for registers

---

1. Program Counter

The **PC** is a 32-bit register containing the address of the instruction being executed.

Normally:

```
PC_next = PC + 4
```

For example:

```
PC = 0x00000000
        ↓
instruction
        ↓
PC = 0x00000004
        ↓
instruction
        ↓
PC = 0x00000008
```

You'll implement this as a register with a clock and reset.

---

2. Instruction Memory

The instruction memory takes the PC as an address:

```
PC ───────► Instruction Memory
                   │
                   ▼
            32-bit instruction
```

For RV32I, instructions are normally **32 bits**.

Example:

```
PC = 0x00000000

Instruction Memory[0] = 32'b...
```

For your FPGA implementation, this can eventually become a **BRAM**.

---

3. Instruction Decoder

A RISC-V instruction contains fields such as:

```
31            25 24   20 19   15 14  12 11     7 6       0
┌───────────────┬───────┬───────┬──────┬───────┬─────────┐
│    funct7     │  rs2  │  rs1  │funct3│  rd   │ opcode  │
└───────────────┴───────┴───────┴──────┴───────┴─────────┘
```

You need to extract:

```
verilog
opcode = instruction[6:0];
rd     = instruction[11:7];
funct3 = instruction[14:12];
rs1    = instruction[19:15];
rs2    = instruction[24:20];
funct7 = instruction[31:25];
```

This is one of the easiest blocks to implement.

---

4. Register File

RISC-V RV32I has:

```
32 registers
×
32 bits
```

So:

```
reg [31:0] registers [0:31];
```

Register:

```
x0
```

is special.

It must **always contain zero**:

```
x0 = 0
```

The register file has:

```
Read port 1 → rs1
Read port 2 → rs2
Write port  → rd
```

Conceptually:

```
 rs1 ─────────►┌─────────────────┐────► data1
 rs2 ─────────►│  Register File  │────► data2
               │     32 × 32     │
 rd ──────────►│                 │
 write_data ──►│                 │
 RegWrite ────►└─────────────────┘
```

5. Immediate Generator

Different RISC-V instruction formats place the immediate in different locations.

You need to support:

```
I-type
S-type
B-type
U-type
J-type
```

For example:

```
asm
ADDI x5, x1, 10
```

The immediate is:

```
10
```

The immediate generator extracts it and sign-extends it to 32 bits.

---

6. ALU

The ALU performs operations such as:

```
ADD
SUB
AND
OR
XOR
SLT
SLTU
SLL
SRL
SRA
```

For example:

```
asm
ADD x5, x1, x2
```

becomes:

```
x1 ────────┐
           ▼
x2 ───► ┌─────┐ ───► result
        │ ALU │
        └─────┘
          ADD
```

7. ALU Control

The main Control Unit doesn't necessarily directly control every ALU operation.

Usually you have:

```
            opcode
              │
              ▼
       ┌──────────────┐
       │ Main Control │
       └──────┬───────┘
            ALUOp
              │
              ▼
funct3 ──►┌─────────────┐◄── funct7
          │ ALU Control │
          └──────┬──────┘
                 ▼
                ALU
```

For example:

```
opcode = OP
funct3 = 000
funct7 = 0000000
```

means:

```
ADD
```

while:

```
opcode = OP
funct3 = 000
funct7 = 0100000
```

means:

```
SUB
```

8. Data Memory

Required for:

```
asm
LW
SW
```

For example:

```
asm
LW x5, 8(x1)
```

The ALU calculates:

```
address = x1 + 8
```

Then:

```
ALU result
     │
     ▼
Data Memory
     │
     ▼
   data
     │
     ▼
Register x5
```

For:

```
asm
SW x5, 8(x1)
```

the data flows the other way:

```
Register x5
     │
     ▼
Data Memory
```

9. Main Control Unit

This is one of the most important blocks.

It looks at the opcode:

```
instruction
     │
     ▼
   opcode
     │
     ▼
┌──────────────┐
│ Control Unit │
└──────────────┘
```

and generates signals such as:

```
RegWrite
ALUSrc
MemRead
MemWrite
MemToReg
Branch
Jump
ALUOp
```

For example:

ADD

```
asm
ADD x5,x1,x2
```

roughly:

```
RegWrite = 1
ALUSrc   = 0
MemRead  = 0
MemWrite = 0
MemToReg = 0
Branch   = 0
Jump     = 0
```

LW

```
asm
LW x5,0(x1)
```

```
RegWrite = 1
ALUSrc   = 1
MemRead  = 1
MemWrite = 0
MemToReg = 1
```

SW

```
asm
SW x5,0(x1)
```

```
RegWrite = 0
ALUSrc   = 1
MemRead  = 0
MemWrite = 1
```

BEQ

```
asm
BEQ x1,x2,label
```

```
RegWrite = 0
ALUSrc   = 0
Branch   = 1
```

10. Multiplexers

MUXes are extremely important in a single-cycle CPU.

For example, the ALU's second input can come from either:

```
rs2
```

or

```
immediate
```

So:

```
               rs2
                │
                ▼
             ┌─────┐
             │ MUX │───► ALU input B
             └─────┘
                ▲
                │
            immediate

              ALUSrc
```

For:

```
asm
ADD x5,x1,x2
```

choose:

```
rs2
```

For:

```
asm
ADDI x5,x1,10
```

choose:

```
immediate
```

11. Branch Unit

For:

```
asm
BEQ x1,x2,label
```

the processor needs to determine:

```
x1 == x2 ?
```

The ALU can perform:

```
x1 - x2
```

and produce:

```
Zero = 1
```

if they are equal.

Then:

```
Branch AND Zero
        │
        ▼
   PC selection
```

12. PC Target Calculation

For a branch:

```
branch_target = PC + immediate
```

For example:

```
asm
BEQ x1,x2,+16
```

the target is calculated using:

```
PC + 16
```

So you'll generally have an adder:

```
      PC
       │
       ├──────────┐
       │       ┌─────┐
       │       │ ADD │
       │       └─────┘
 immediate ───────┤
                  │
                  ▼
            Branch Target
```

13. PC Selection MUX

Finally, the processor has to decide where the next instruction comes from.

At minimum:

```
PC + 4
```

or:

```
Branch Target
```

For a more complete RV32I implementation, also:

```
JAL target
JALR target
```

So conceptually:

```
             PC + 4
               │
        Branch Target
               │
           JAL Target
               │
           JALR Target
               ▼
           ┌────────┐
           │ PC MUX │
           └───┬────┘
               ▼
               PC
```

The complete basic architecture

For the **RV32I single-cycle CPU you're working toward**, I'd structure it like this:

```
                 ┌──────────┐
                 │    PC    │
                 └────┬─────┘
                      ▼
              ┌─────────────────┐
              │ Instruction Mem │
              └───────┬─────────┘
                 instruction
       ┌──────────────┼──────────────┐
       ▼              ▼              ▼
  ┌─────────┐    ┌──────────┐   ┌───────────┐
  │ Control │    │ Register │   │ Immediate │
  │  Unit   │    │   File   │   │ Generator │
  └────┬────┘    └────┬─────┘   └─────┬─────┘
       │         rs1/rs2 data         │
       │              └───────┬───────┘
       │                      ▼
       │                 ┌─────────┐
       └────────────────►│   ALU   │
                         └────┬────┘
                          ALU result
                     ┌────────┴────────┐
                     ▼                 ▼
              ┌─────────────┐   ┌───────────┐
              │ Data Memory │   │ Writeback │
              └──────┬──────┘   │    MUX    │
                     │          └─────┬─────┘
                     └────────────────┤
                                      ▼
                                 Register File

     PC ────────────► PC + 4
      └──────────► Branch/Jump Target
                          ▼
                     ┌────────┐
                     │ PC MUX │
                     └───┬────┘
                         ▼
                         PC
```

Recommended development order

Since you previously wanted to **build and test each block before moving to the next**, don't start by writing the entire CPU.

Build it in this order:

```
1. PC
      ↓
2. Instruction Memory
      ↓
3. Instruction Decoder
      ↓
4. Register File
      ↓
5. Immediate Generator
      ↓
6. ALU
      ↓
7. ALU Control
      ↓
8. Main Control Unit
      ↓
9. Data Memory
      ↓
10. Write-back MUX
      ↓
11. Branch/Jump Unit
      ↓
12. PC selection
      ↓
13. Complete CPU
```

And test instructions progressively:

```
ADD
SUB
   ↓
ADDI
AND / OR / XOR
   ↓
LW / SW
   ↓
BEQ / BNE
   ↓
JAL
   ↓
JALR
   ↓
Full RV32I subset
```

For your **first FPGA implementation**, I'd strongly recommend starting with a small **RV32I subset** rather than trying to implement all of RISC-V immediately:

```
ADD
SUB
AND
OR
XOR
ADDI
LW
SW
BEQ
JAL
```

Once those work in simulation, adding the remaining instructions becomes much easier.

> it should be able to run a hex file

Yes. If your goal is a **RISC-V single-cycle CPU that can run a `.hex` program file on your FPGA**, then we should design the architecture around a **preloaded instruction memory**.

The important distinction is that the CPU doesn't directly "run a hex file." The **HEX file is loaded into instruction memory**, and the CPU fetches the 32-bit instructions from that memory.

Overall system

```
          program.hex
               │
               ▼
      ┌─────────────────┐
      │ Instruction RAM │
      │                 │
      │ 00000013        │
      │ 00100093        │
      │ 00208133        │
      │ ...             │
      └───────┬─────────┘
              │ instruction
              ▼
         ┌─────────┐
         │   CPU   │
         │         │
         │ RISC-V  │
         └────┬────┘
              ▼
         Data Memory
              │
              ▼
          Registers
```

For an FPGA, the simplest approach is:

`.hex` → **initialized BRAM/register array** → **RISC-V CPU**

Architecture I'd recommend

For your project, build this:

```
                  ┌──────────┐
                  │    PC    │
                  │  32-bit  │
                  └────┬─────┘
                       ▼
              ┌──────────────────┐
              │Instruction Memory│
              │                  │
              │ 32-bit words     │
              │ program.hex      │
              └───────┬──────────┘
                      ▼
              32-bit instruction
       ┌──────────────┼───────────────┐
       ▼              ▼               ▼
  ┌─────────┐   ┌──────────┐   ┌───────────┐
  │ Control │   │ Register │   │ Immediate │
  │  Unit   │   │   File   │   │ Generator │
  └────┬────┘   └────┬─────┘   └─────┬─────┘
       │          rs1 / rs2          │
       │             └───────┬───────┘
       │                     ▼
       │                ┌─────────┐
       └───────────────►│   ALU   │
                        └────┬────┘
                         ALU result
                 ┌───────────┴───────────┐
                 ▼                       ▼
           ┌──────────┐             ┌───────┐
           │ Data RAM │             │  MUX  │
           └────┬─────┘             └───┬───┘
                └───────────────────────┤
                                        ▼
                                  Register File
```

And separately:

```
         PC
          │
   ┌─────────────┐
   │ PC Target   │
   │ Calculator  │
   └──────┬──────┘
          ▼
     ┌────────┐
     │ PC MUX │
     └───┬────┘
         ▼
         PC
```

How the HEX file works

Suppose your `program.hex` contains:

```
00000013
00100093
00208133
003101B3
```

Each line can represent one 32-bit RISC-V instruction.

Your Verilog instruction memory can be something like:

```
verilog
module instruction_memory (
    input  wire [31:0] address,
    output wire [31:0] instruction
);

    reg [31:0] memory [0:1023];

    initial begin
        $readmemh("program.hex", memory);
    end

    assign instruction = memory[address[11:2]];

endmodule
```

The important part is:

```
verilog
$readmemh("program.hex", memory);
```

That loads the HEX file into the memory array.

And:

```
verilog
memory[address[11:2]]
```

converts the byte address from the PC into a **word address**.

For example:

```
PC = 0
      ↓
memory[0]

PC = 4
      ↓
memory[1]

PC = 8
      ↓
memory[2]

PC = 12
      ↓
memory[3]
```

What happens when the CPU starts?

Suppose:

```
PC = 0
```

The CPU does:

```
PC
│
▼
Instruction Memory
│
▼
program.hex
│
▼
instruction 0
│
▼
Decode
│
▼
Execute
│
▼
Write result
│
▼
PC = PC + 4
```

Because it's a **single-cycle CPU**, all of that happens during one clock cycle.

Next cycle:

```
PC = 4
```

and it fetches the second instruction.

You should also have Data Memory

If you want programs such as:

```
asm
addi x1, x0, 10
addi x2, x0, 20
add  x3, x1, x2
sw   x3, 0(x0)
```

you need:

```
            CPU
             │ address
             ▼
       ┌─────────────┐
       │ Data Memory │
       │             │
       │    RAM      │
       └─────────────┘
```

This is separate from instruction memory.

So I'd use:

```
Instruction Memory
        │ instructions
        ▼
       CPU
        │ address/data
        ▼
   Data Memory
```

For your MicroZed

Since you're working with the **MicroZed 7010**, you have a very nice option.

You can initially make the entire processor in the **PL FPGA fabric**:

```
              MicroZed
        ┌────────────────────┐
        │                    │
        │      Zynq-7010     │
        │                    │
        │   ┌─────────────┐  │
        │   │ RISC-V CPU  │  │
        │   │             │  │
        │   │  PC         │  │
        │   │  Reg File   │  │
        │   │  ALU        │  │
        │   │  Control    │  │
        │   │  IMEM       │  │
        │   │  DMEM       │  │
        │   └─────────────┘  │
        │          │         │
        │          ▼         │
        │         GPIO       │
        └──────────┬─────────┘
                   │
                  LEDs
```

Initially, don't even involve the ARM processor.

Get the RISC-V CPU working entirely in the FPGA.

A very good first milestone

Make a CPU that can execute this:

```
asm
addi x1, x0, 5
addi x2, x0, 10
add  x3, x1, x2
```

Your HEX file would contain the machine-code equivalents:

```
00500093
00A00113
002081B3
```

Then after execution:

```
x1 = 5
x2 = 10
x3 = 15
```

You can connect `x3` to LEDs or an ILA to verify it.

Then move to:

```
asm
sw x3, 0(x0)
lw x4, 0(x0)
```

Then:

```
asm
beq x1, x2, label
```

Then jumps.

One important thing

If by **"run a hex file"** you mean:

> "I want to compile a C program, produce a `.hex`, put it on the FPGA, and have my RISC-V CPU execute it"

Yes, absolutely.

The eventual workflow can be:

```
C program
    ↓
RISC-V GCC
    ↓
ELF
    ↓
HEX
    ↓
program.hex
    ↓
FPGA Instruction Memory
    ↓
Your RISC-V CPU
    ↓
Program executes
```

We can build this incrementally, and I would suggest we start with **`PC → Instruction Memory → Decoder → Register File → ALU`**, test each independently, and then connect them into the complete single-cycle CPU.

Printed using ChatGPT to PDF, powered by PDFCrowd HTML to PDF API.